Analyzing the Development Lifecycle of Vichan

Tristan Lopez, Noah Cash, Jonathan Lange, and Ar-Raniry Ar-Rasyid

Objective

Vichan is a free, open-source program used to run imageboard websites. Its source code has been publicly available on GitHub since 2010, and the project formally ended in 2026. The goal of this project is to study the complete history of changes made to Vichan in order to understand how the work on it evolved over time. Specifically, we aim to answer two questions. First, as the project matured, did its developers shift away from adding new features and toward spending most of their effort on security and bug fixes? Second, how much of the work was driven by outside problems, such as newly discovered security vulnerabilities, as opposed to the team's own decisions to reorganize or improve existing code?

Motivation

A large share of open-source software is built and maintained by volunteers, and many of these projects eventually slow down or stop entirely. Understanding how the work on such projects changes as they age can help explain why they decline and what it takes to keep them running. Vichan is a strong case study because its entire lifespan, from its first change in 2010 to its end in 2026, is publicly recorded. Reviewing thousands of changes by hand would be slow and prone to inconsistency, so this project uses automated classification to make the question practical to answer while still allowing the results to be checked for accuracy.

Data

The data for this project comes from the official Vichan repository on GitHub (https://github.com/vichan-devel/vichan). A repository stores every saved change to the code, known as a commit. Each commit records who made the change, when it was made, a short message describing it, and the exact lines of code that were added or removed. We will obtain this data through repository mining: we will download a complete copy of the repository using Git and write a Python script that extracts the history into structured CSV files.

A preliminary download shows that the repository contains approximately 9,600 commits. These commits modify files about 25,000 times and contain roughly 65,000 individual blocks of changed code. We will treat each block of changed code as one record, which means we expect to process approximately 65,000 records.

We anticipate several challenges with this data. Many commit messages are vague, often consisting of a single word such as "fix" or "update," so we will also examine the files and lines that changed to understand each commit's purpose. In addition, the same developer often appears under different names and email addresses; one developer alone used at least six. These identities must be combined to avoid counting a single person as several. Finally, because our labels will be assigned automatically, some will be incorrect, so the results must be validated before they can be trusted.

Approach

This project focuses on the retrieval, cleaning, and analysis stages of the Digital Archaeology process. We will begin by writing a Python script that downloads the repository's history and saves it as CSV files. Next, we will clean the data by removing merge commits (which only

combine work that already exists elsewhere in the history), converting all timestamps to a single time zone, and combining the different names that belong to the same developer.

We will then classify each change into one of five categories: Feature, Bug Fix, Security Fix, Code Cleanup, or Documentation. Classification will begin with keyword matching; for example, a message containing "XSS" or "vulnerability" will be labeled as a security fix. Changes that keywords cannot classify will be labeled using an AI language model. To measure accuracy, we will label a small random sample by hand and compare those labels with the automated results. Finally, we will count the number of changes in each category per year and present the results as graphs, which will show whether security and bug-fix work grew as a share of total activity over the project's lifetime. The main tools will be Git, Python, the pandas library, an AI language model, and Matplotlib for visualization.

Team Responsibilities

Jonathan Lange will write the script that collects the repository data and saves it in a usable format. Tristan Lopez will be responsible for cleaning the data, including removing merge commits and combining duplicate developer identities. Ar-Raniry Ar-Rasyid will build the automated classification process and lead the accuracy check against the hand-labeled sample. Noah Cash will produce the graphs and lead the writing of the final report.

Timeline and Milestones

Dates	Milestone
October 22 – October 26	Repository data collected and saved as CSV files
October 27 – October 30	Data cleaned and developer identities combined
October 31 – November 3	Classification tested on a small sample
November 4 – November 8	Full dataset classified and accuracy checked
November 9 – November 12	Analysis and graphs completed
November 13 – November 16	Final report and presentation completed
November 17	Project presentation

Expected Outcome

The final result will be a data analysis, supported by graphs, that describes how the work on Vichan changed over its lifetime. We expect it to show whether the project moved from building new features toward primarily security and maintenance work, and to estimate how much of the project's activity was a response to outside problems rather than internal improvement.